https://github.com/SETAPESU26/23_dots_and_boxes
this is the link to a game.
I'm doing a university lab on vibe coding. The repo is https://github.com/SETAPESU26/23_dots_and_boxes (a Python Dots and Boxes game with board.py, rules.py, game.py, main.py). If you can't open it, here are the files:

[paste README.md, main.py, game.py, board.py, rules.py here]

Task: the README says the game has a bug around box completion, score and turns. Please:
1. Read the code carefully and run through scenarios where a move completes one box and two boxes at once, then tell me honestly whether score and turn handling are correct.
2. Find every real bug or crash. I already noticed that the board drawing has no dots at the corners and the rows are different widths, that typing "H 0 ²" crashes with a ValueError, and that Ctrl+D crashes with EOFError.
3. Fix them without replacing the modular structure (board.py, rules.py, game.py, main.py stay separate). Give me the full updated files and explain each fix briefly.
Now add a substantial gameplay feature that touches more than one module. Add all of these:
1. A play-against-the-computer mode in a new ai.py (take a box if one is available, otherwise avoid giving the opponent a third side), started with "python main.py --computer".
2. An undo command ("U") that restores the board, score and turn. In computer mode it should go back to my last move.
3. Board size options --rows and --cols (1 to 9).
Keep the default "python main.py" as the original 2x2 two-player game, keep the code modular, don't put all new logic in main.py, and use only the standard library. Give me the full updated files.
Strengthen input and game-state handling. The program must safely handle malformed commands, invalid or negative coordinates, non-ASCII digits, very long numbers, repeated lines, and moves made after the board is already complete. Invalid input must never corrupt the board or the score, and Ctrl+D or Ctrl+C must end the game cleanly. Put the validation in rules.py and the board, not in main.py. Give me the full updated files.
Write automated tests with unittest (standard library only) in test_dots_and_boxes.py. They must include at least: a valid horizontal move, a valid vertical move, an invalid/repeated move, completion of a box, and the end-of-game condition. Also cover completing two boxes at once, a move after game over, malformed input, undo and the computer player.
Then write CHANGES.md documenting the bugs fixed, the new features and the design decisions. Run the tests mentally and fix anything that would fail, and give me the final files.

I was able to access your actual GitHub repository and read README.md, main.py, game.py, board.py, and rules.py. I've prepared a complete modular update, added ai.py, automated tests, and CHANGES.md, and run the test suite successfully.

**One important finding:** the original code's score and turn logic already appears correct. It calculates the difference between completed boxes before and after a move, awards the correct number of points, and retains the turn when a player completes a box. A move that completes two boxes should award two points and retain the turn. I wouldn't claim that this particular bug was reproduced in the current version of the repository.

The update focuses on the real weaknesses I could verify: unsafe direct board mutation, malformed and Unicode numeric input, missing corner dots, command-line configuration, undo, computer play, and automated regression tests.

**1. Get the complete updated project**

**Complete project ZIP**

All Python modules, tests, change log and test results.

Download the complete project

**Individual files**

board.py

Board state, safe line placement, rendering and snapshots.

Open file

rules.py

Input parsing, coordinate validation and box counting.

Open file

game.py

Game loop, turns, scores, undo and clean interruption handling.

Open file

main.py

Command-line arguments and game startup.

Open file

ai.py

Rule-based computer opponent.

Open file

test_dots_and_boxes.py

10 automated unit tests using unittest.

Open file

CHANGES.md

Bugs, features and design decisions.

Open file

**2. What changed?**

| **File** | **Main changes** |
|---|---|
| board.py | Draws every dot, supports rectangular boards, protects against invalid moves, and supports snapshots. |
| rules.py | Rejects negative/out-of-range coordinates, repeated edges, non-ASCII digits, malformed commands and moves after game over. |
| game.py | Adds undo, restores scores and turns, supports computer play, and handles Ctrl+D/Ctrl+C. |
| ai.py | Takes available boxes first; otherwise avoids creating three-sided boxes where possible. |
| main.py | Adds --computer, --rows and --cols while preserving the default game. |
| test_dots_and_boxes.py | Covers legal moves, invalid moves, one/two-box completion, game over, malformed input, undo and AI behavior. |

**3. Run the game**

Extract the ZIP and open a terminal in the project directory.

# Original two-player game

python main.py

# Play against the computer

python main.py --computer

# Custom rectangular board

python main.py --rows 3 --cols 5

# Custom board against the computer

python main.py --computer --rows 3 --cols 5

# Run automated tests

python -m unittest -v test_dots_and_boxes.py

During a game, enter moves such as H 0 1 or V 1 0. Enter U to undo your previous move. In computer mode, undo restores the state before your last move, including the computer's reply.

**4. Test results**

**10 tests passed**

Executed with Python's unittest runner; exit code 0.

Horizontal and vertical moves

Invalid and repeated moves

One-box and two-box completion

Game-over protection

Malformed and Unicode input

Undo and computer move selection

Board-size validation

One distinction worth keeping in mind: the tests were executed, but they do not yet simulate every interactive terminal scenario. In particular, the computer opponent is deliberately a simple heuristic, not an optimal strategy, and I would recommend one manual check of undo after a computer capture before submitting.

For your university lab, keep the original repository unchanged in a separate branch or commit before replacing files. That will help you demonstrate the before-and-after behavior and explain which changes you made.

in the before behavior i want to show the error as well soo tell me what moves i need to make

For your lab demo, the best way to show the **before behavior** is to run the original game and make moves that demonstrate scoring, turn handling, and the input crashes. Then repeat the same scenarios with the fixed version.

**1. Demonstrate the box-completion behavior**

Start the **original version**:

python main.py

Use a 2×2 board. Each box needs four edges: top, bottom, left and right.

**Scenario A — Complete one box**

Enter these moves in order, switching players as indicated by the game:

**1**

**Player 1:** H 0 0

Top edge of the top-left box.

**2**

**Player 2:** H 1 0

Bottom edge of the top-left box.

**3**

**Player 1:** V 0 0

Left edge of the box.

**4**

**Player 2:** V 0 1

Right edge; completes the box.

**Expected result:** Player 2 gets one point and keeps their turn. If this works in the original, don't claim that single-box scoring is broken.

**Scenario B — Complete two boxes at once**

This is the more interesting scoring test. Use the original 2×2 board and enter:

| Move | Command | Purpose |
|---|---|---|
| 1 | H 0 0 | Top of top-left box |
| 2 | H 0 1 | Top of top-right box |
| 3 | H 1 0 | Bottom of top-left box |
| 4 | H 1 1 | Bottom of top-right box |
| 5 | V 0 0 | Left of top-left box |
| 6 | V 0 2 | Right of top-right box |
| 7 | V 0 1 | Shared edge; completes both boxes |

**Expected result:** the player making move 7 receives **2 points** and gets another turn.

Watch the displayed score and player number. If both are correct, the original implementation handles this scenario correctly too.

**2. Demonstrate the actual bugs you mentioned**

Run each test separately in the original version.

**Bug 1 — Missing corner dots / uneven rows**

Start python main.py and take a screenshot of the initial board. Point out the missing corner dots or misaligned rows.

Expected fix: all dot intersections should be present and the board should align.

**Bug 2 — Superscript digit crashes input**

At a move prompt, type this exactly:

H 0 ²

Expected before: ValueError crash if the original parser passes this character to int(). After the fix: a harmless invalid-move message.

**Bug 3 — Ctrl+D crashes**

Restart the original game and press **Ctrl+D** at the input prompt in a Linux/macOS terminal, or in the equivalent terminal environment you use for the lab.

Expected before: an unhandled EOFError, if the original loop does not catch it. After the fix: the game exits cleanly.

**Bug 4 — Repeated or invalid moves**

Try:

H 0 0

H 0 0

H -1 0

H 0 999999999999999999

Record whether the original rejects these cleanly, crashes, or alters the board incorrectly. Don't claim a state-corruption bug unless you observe it.

**3. Demonstrate the new features after the fix**

Run the updated version:

python main.py --computer --rows 2 --cols 2

Then demonstrate:

1. Make a valid move.
2. Enter U and show that the move is undone.
3. Complete a box and verify the score.
4. Try H 0 ² and show that the game continues.
5. Try a repeated move and show that the board remains unchanged.

Finally, run the tests:

python -m unittest -v test_dots_and_boxes.py

**For your presentation:** distinguish between *reproduced original bugs* and *new features*. The two-box scoring scenario is a regression check, not necessarily an original bug. That makes your demonstration more credible: you're showing exactly what was broken and proving that the fix preserves behavior that was already correct.

do the deliverables mention which ai i need to use for vibecoding

No — the deliverables you shared **don't specify which AI tool you must use for vibe coding**.

They specify what you need to implement and submit:

- The updated modular Python files.

- The computer opponent and undo feature.
- Input validation and crash fixes.
- Automated tests in test_dots_and_boxes.py.
- CHANGES.md documenting the changes.

They don't explicitly name tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or Gemini.

**One thing to check:** your university lab may have a separate rubric or instruction document specifying which vibe-coding tool to use, or whether you need to submit evidence of your AI-assisted development process.

If you share that rubric, I can check it for any tool requirements.

**Lab_4_VibeCoding_Student_handout (1).pdf**PDF

I checked your Lab 4 handout. **You can use ChatGPT, Gemini, or Claude — the university does not mandate a specific AI tool.**

The handout explicitly lists the following under *Software Requirements*:

Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Lab_4_VibeCoding_Student_handout (1).pdf

**What you need to submit**

According to the handout, upload these to your lab repository under the Lab-4 folder:

Lab_4_VibeCoding_Student_handout (1).pdf

- **Before and after videos:** 10 seconds each.
- **Updated code:** Push it to your personal repository.
- **Chat history:** Export your AI conversation as a DOC or PDF.

You must also work individually, and you must not raise a pull request to the original SETAPESU26 repository.

Lab_4_VibeCoding_Student_handout (1).pdf

Lab_4_VibeCoding_Student_handout (1).pdf

**My recommendation:** Since you've already worked through the bug fixes and feature requests with ChatGPT, you can continue using ChatGPT. Just make sure you export the conversation for submission and record the original game before changing your repository.